FIAP • SPRINT 4

MOVE POINTS

Documentação da API REST

Domain Driven Design Using Java

Solução de mobilidade sustentável com registro de viagens, cálculo de pontos e ranking. Projeto em Java com Spring Boot.

EQUIPE / DEVWARRIORS

André Luiz Ramos Forastieri
RM 572203 | 1TDSPH

Guilherme de Oliveira Alves
RM 571683 | 1TDSPI

Fabrício Torres Haine
RM 573332 | 1TDSPY

Leonardo Pinto Neto
RM 571960 | 1TDSPY

A proposta do MovePoints

01 / Objetivo, escopo e evolução da solução

Objetivo do projeto

O MovePoints incentiva o uso de ônibus, metrô, trem e bicicleta. Os usuários registram deslocamentos e acumulam pontos proporcionais à distância percorrida e ao modal de transporte. O saldo é mantido por usuário e um ranking destaca os participantes.

Evolução da Sprint 3 para a Sprint 4

A solução evolui de uma aplicação de console para uma API RESTful com Spring Boot, integrada ao front-end, com operações CRUD, camadas modelo/DAO/BO/Resource e tratamento de exceções. O projeto prevê deploy no Render, mas as capturas recebidas não comprovam a publicação.

ÔNIBUS 1,0 pt/km | **METRÔ** 1,2 pt/km | **TREM** 1,2 pt/km | **BICICLETA** 2,0

Principais funcionalidades

Cadastro de usuários
Nome e CPF único; duplicidade gera erro 400.

Registro de viagens
Modal, distância e pontos calculados automaticamente.

Atualização de viagens
Recalcula pontos e ajusta o saldo acumulado.

Exclusão com cascata
Excluir usuário também remove as viagens vinculadas.

Ranking e fatores
Classificação por pontos e consulta dos fatores de pontuação.

Validações HTTP
Respostas 400, 404 e 500 em JSON padronizado.

Endpoints da API REST

02 / Operações disponíveis no serviço Java

Verbo	URI	Descrição	HTTP
GET	/usuarios	Listar usuários	200
GET	/usuarios/{id}	Consultar usuário	200 / 404
POST	/usuarios	Cadastrar usuário	201 / 400
PUT	/usuarios/{id}	Atualizar nome/CPF	200 / 400 / 404
DELETE	/usuarios/{id}	Excluir usuário e viagens	204 / 404
GET	/viagens	Listar viagens	200
GET	/viagens/{id}	Consultar viagem	200 / 404
GET	/viagens/usuario/{usuarioId}	Viagens por usuário	200 / 404
GET	/viagens/fatores	Fatores de pontuação	200
POST	/viagens	Registrar viagem e pontuar	201 / 400 / 404
PUT	/viagens/{id}	Editar viagem e saldo	200 / 400 / 404
DELETE	/viagens/{id}	Remover viagem e debitar	204 / 404
GET	/ranking	Ranking de pontuação	200

Tratamento de erros e CORS

A API utiliza respostas HTTP de sucesso e erros padronizados (400 para dados inválidos, 404 para recursos não encontrados e 500 para falhas internas). A classe CorsFilter permite GET, POST, PUT, DELETE e OPTIONS para consumo pelo front-end.

Exemplos de integração

03 / Requisições e respostas JSON

POST /usuarios • 201 Created

SQL / JAVA

REQUISIÇÃO

```
{ "nome": "Maria Souza",
  "cpf": "111.222.333-44" }
```

RESPOSTA

```
{ "id": 5, "nome": "Maria Souza",
  "cpf": "111.222.333-44",
  "pontosTotal": 0.0 }
```

POST /viagens • 201 Created

SQL / JAVA

REQUISIÇÃO

```
{ "usuarioId": 5,
  "tipoTransporte": "bicicleta", "distanciaKm": 10 }
```

RESPOSTA

```
{ "id": 1, "usuarioId": 5,
  "tipoTransporte": "bicicleta", "distanciaKm": 10.0,
  "pontos": 20.0, "dataViagem": "2026-10-27" }
```

Exceções previstas

400 Bad Request

Modal inválido. Opções válidas: onibus, metro, trem, bicicleta.

404 Not Found

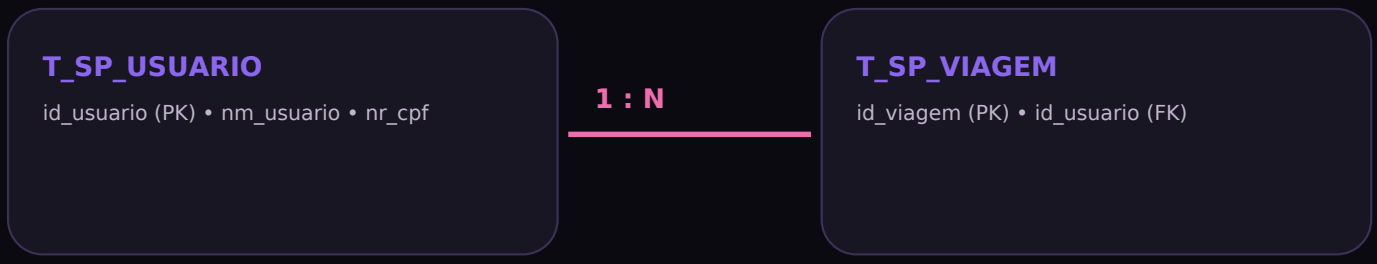
Usuário não encontrado com o ID: 99999.

Persistência em Oracle

04 / Entidades, relacionamento e criação

Modelo entidade-relacionamento (MER)

O modelo Java utiliza duas entidades em uma relação 1:N: um usuário pode registrar várias viagens, enquanto cada viagem pertence a um único usuário. A chave estrangeira de viagens utiliza ON DELETE CASCADE. O MER abaixo é descrito a partir do SQL entregue, não é uma imagem exportada da ferramenta de modelagem.



Script de criação resumido

SQL / JAVA

```
CREATE SEQUENCE seq_usuario START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_viagem START WITH 1 INCREMENT BY 1;

CREATE TABLE T_SP_USUARIO (
  id_usuario NUMBER(10) PRIMARY KEY,
  nm_usuario VARCHAR2(100) NOT NULL,
  nr_cpf VARCHAR2(14) NOT NULL UNIQUE,
  qt_pontos NUMBER(10,2) DEFAULT 0
);

CREATE TABLE T_SP_VIAGEM (
  id_viagem NUMBER(10) PRIMARY KEY,
  id_usuario NUMBER(10) NOT NULL,
  ds_tipo_transporte VARCHAR2(20) NOT NULL,
  nr_distancia_km NUMBER(10,2) NOT NULL,
  nr_pontos NUMBER(10,2) NOT NULL,
  dt_viagem DATE NOT NULL,
  FOREIGN KEY (id_usuario) REFERENCES
    T_SP_USUARIO(id_usuario) ON DELETE CASCADE
);
```

Execução e arquitetura

05 / Ambiente de desenvolvimento e instruções

Ferramentas

Ferramenta	Configuração informada
Java JDK	21 LTS ou superior; testado com 21 e 24
Maven	Wrapper (mvnw) incluído no projeto
IDE	IntelliJ IDEA, VS Code ou Eclipse
Oracle	Servidor FIAP: oracle.fiap.com.br

Como executar localmente

SQL / JAVA

```
cd "Challenge Sprint 4"

# Windows
./mvnw.cmd spring-boot:run

# Linux / macOS
./mvnw spring-boot:run

# Endpoints de teste
http://localhost:8080/usuarios
http://localhost:8080/ranking
```

IntelliJ e inicialização do banco

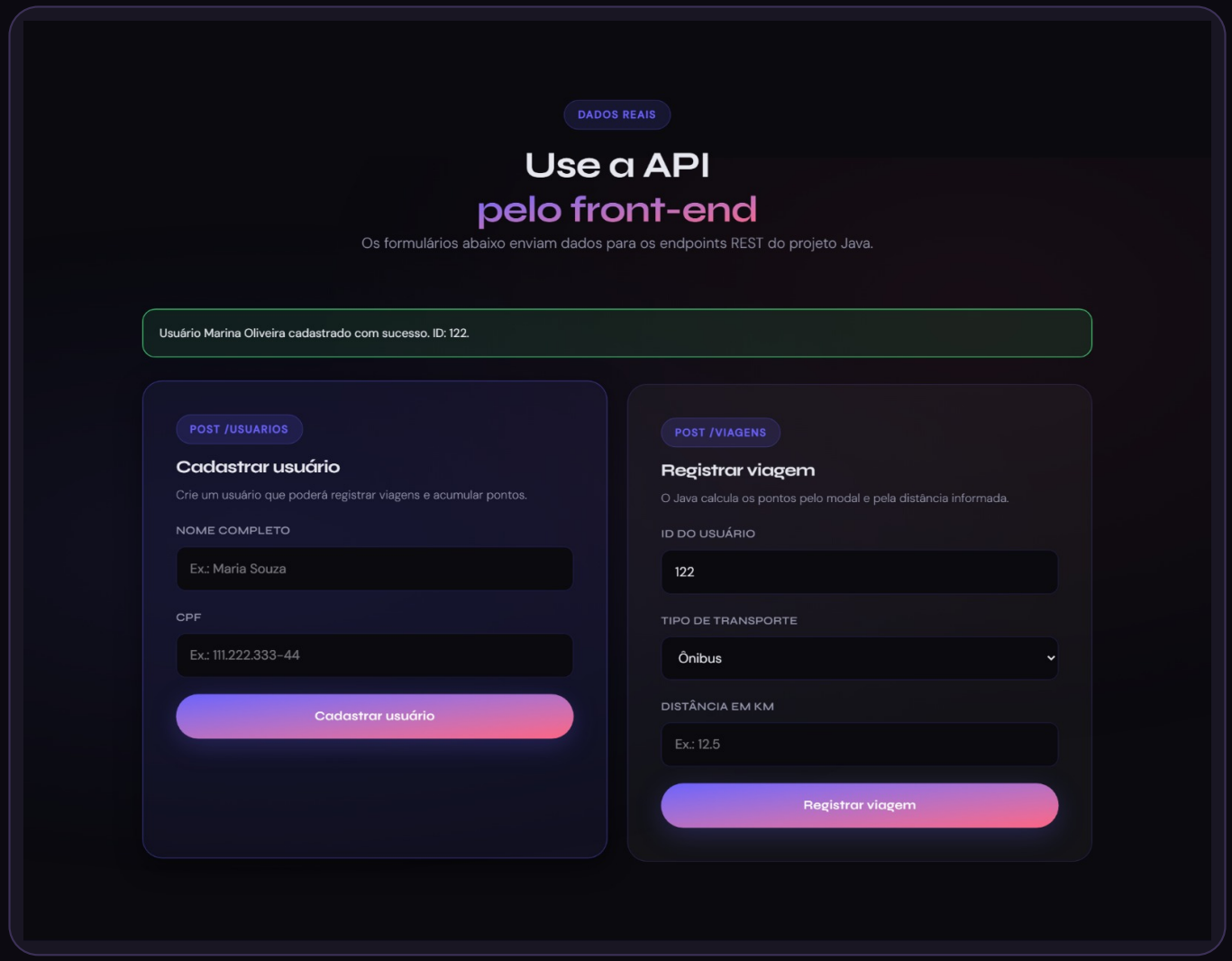
Abra a pasta do projeto pelo menu File → Open, importe o pom.xml e execute DevwarriorApplication.java. Segundo a documentação recebida, InicializadorBanco e ConnectionFactory.inicializarBanco() criam as tabelas e sequências no primeiro início. O SQL completo está em sql/script_criacao_tabelas.sql.

Variáveis de ambiente

DB_HOST: oracle.fiap.com.br DB_USER: RM573332 DB_PASSWORD: senha do Oracle FIAP PORT: 8080 (ou porta disponibilizada pela hospedagem).

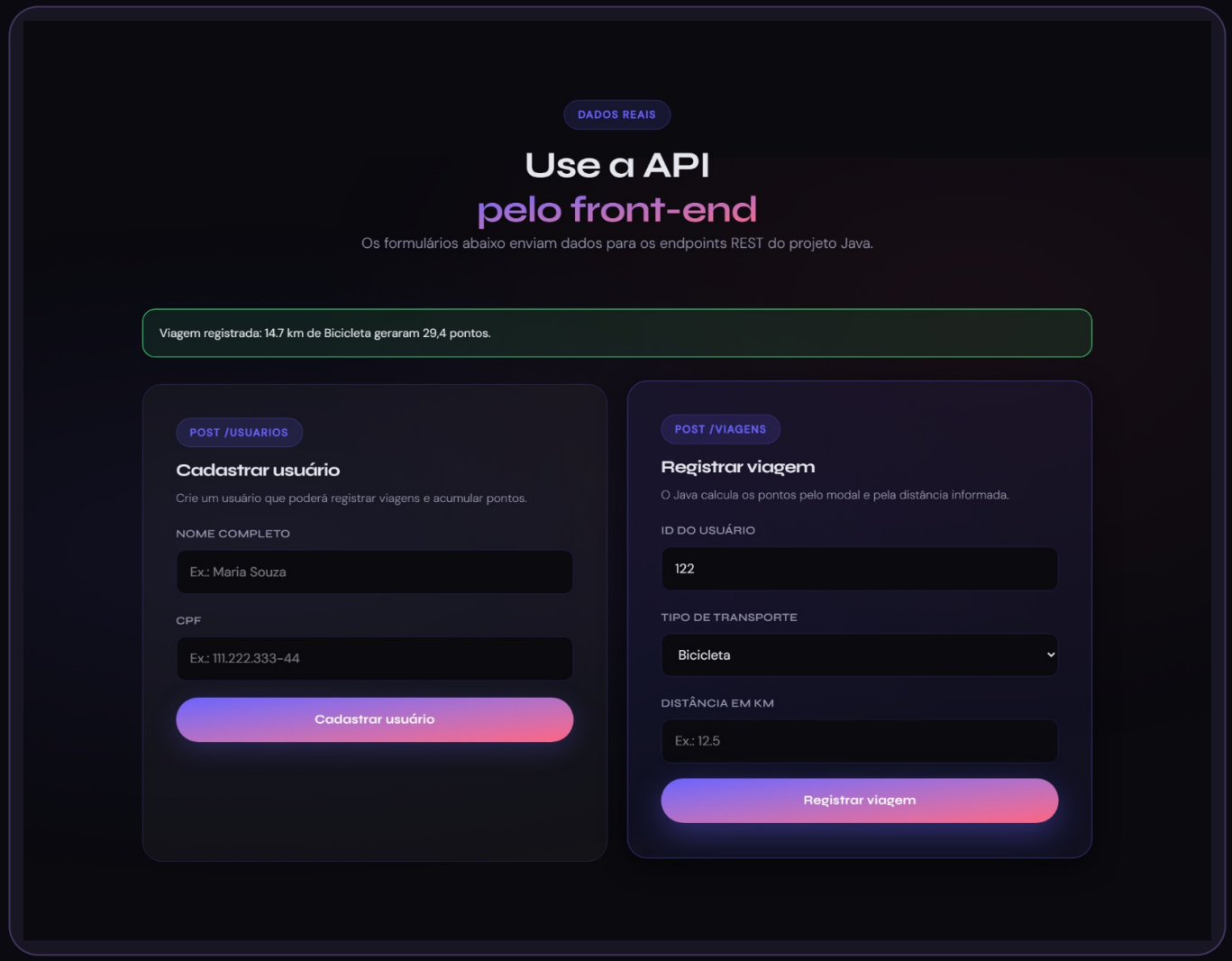
Cadastro de usuário

Confirmação do cadastro de Marina Oliveira (ID 122).



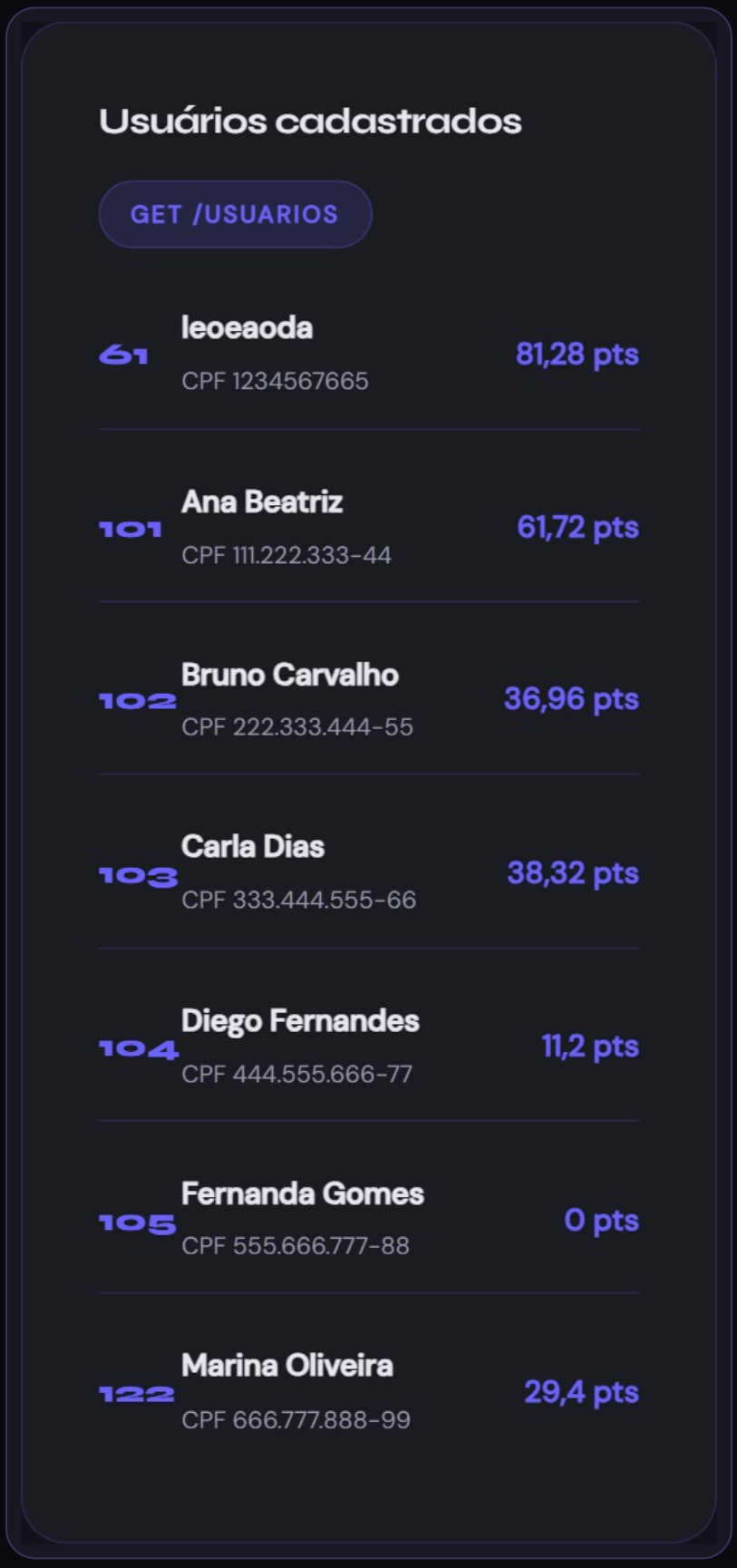
Registro de viagem

Bicicleta: 14,7 km e 29,4 pontos apresentados.



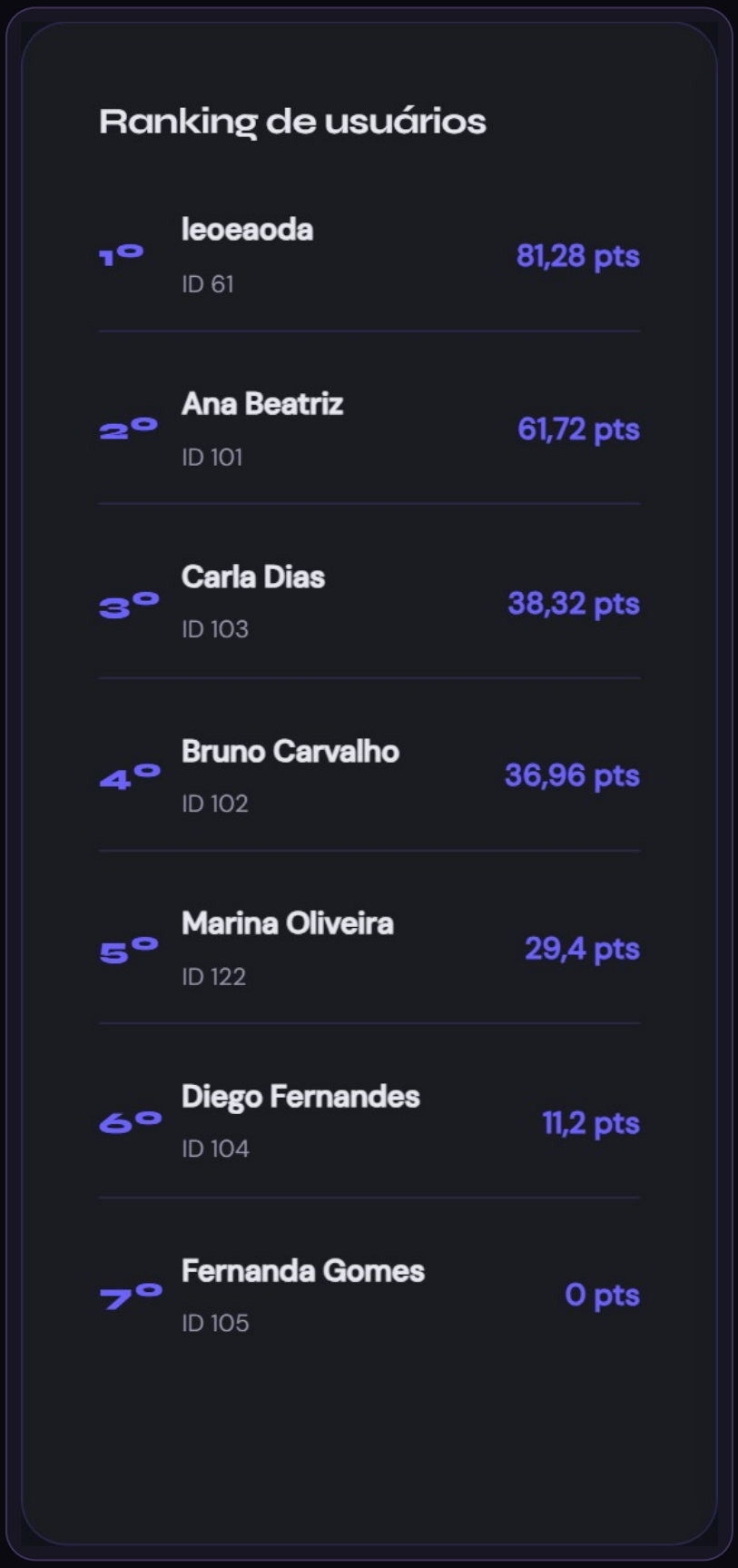
Usuários cadastrados

Listagem com identificação e pontuação.



Ranking de usuários

Classificação por pontos acumulados.



Página institucional

Página do projeto; não comprova deploy no Render.

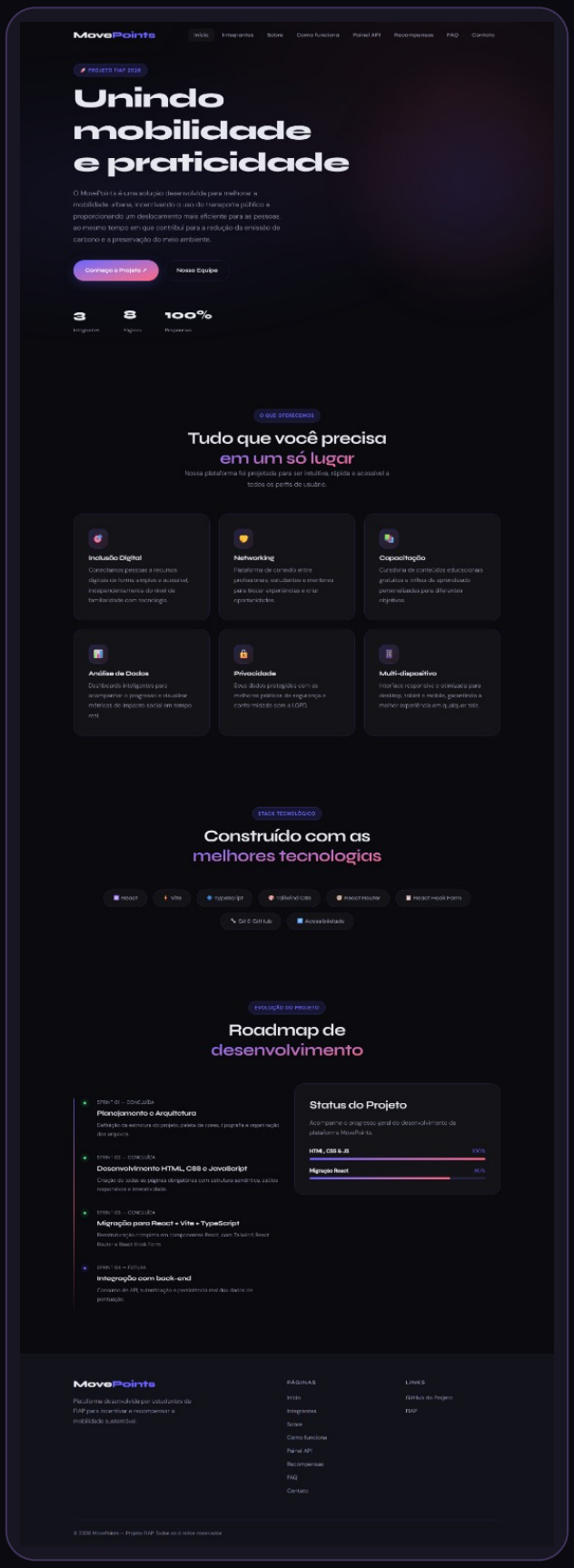


Diagrama de classes

Arquitetura de Resources, BOs, DAOs e entidades.

